OOP Final Project

Real-World Management Application

Project Requirements & Guidelines

Instant

Credits
Mohamed Abu Obaida

Python • Object-Oriented Programming • File Handling • GUI

1. Project Objective

In this project, students will work in a team of 2–3 students to design and develop a small real-world management application using Python and Object-Oriented Programming (OOP). The goal is to apply OOP concepts to a complete application rather than isolated exercises.

• OOP classes and objects

• Constructors

• Encapsulation

• Inheritance

• Polymorphism

• Abstraction

• Composition

• Type hints

• Exception handling

• File handling

• GUI

GUI development is part of the project and is a self-study topic. Students are expected to research and learn the GUI framework they choose.

2. Team Rules

Team Size: 2–3 students per team.

Each student must contribute to the project. During the final presentation, any team member may be asked to explain any class, method, OOP concept, file handling, GUI implementation, or business rule. Dividing the project does not mean dividing the knowledge.

3. Choose ONE Project

• 1. Gym Management System

• 2. Library Management System

• 3. Academy Courses Management System

• 4. Restaurant Management System

• 5. Clinic Management System

• 6. Inventory Management System

• 7. Car Rental Management System

• 8. Hotel Management System

• 9. Personal Finance Management System

• 10. Small E-Commerce System

• 11. Cinema Management System

• 12. Car Service / Workshop System

• 13. Event Management System

• 14. Pharmacy Management System

• 15. Task & Project Management System

Teams may propose another small real-world application for approval.

4. General Application Requirements

Classes

Create at least 5 meaningful classes. Classes should represent real concepts in the chosen application.

CRUD

Support appropriate Create, Read, Update and Delete operations.

Search

Provide meaningful search functionality such as search by name, ID, title or category.

Filtering

Provide useful filters such as Active/Inactive, Paid/Unpaid, Available/Unavailable or Completed/Pending.

Business Rules

Implement real business logic rather than only CRUD operations.

Validation

Validate required fields, IDs, quantities, prices, dates and other relevant input.

Error Handling

Handle expected errors gracefully and display meaningful messages.

5. OOP Requirements

OOP Concept	Requirement
Classes	At least 5 meaningful classes
Objects	Create and use objects
Constructors	Use __init__
Encapsulation	Private attributes + controlled access
Properties	Use @property where appropriate
Inheritance	At least one meaningful inheritance hierarchy
Polymorphism	Same interface with different implementations
Abstraction	Use an abstract base class where appropriate
Composition	Classes should contain/use other objects
Type Hints	Type hints throughout the project
Exception Handling	Handle expected errors
Methods	Business behavior belongs to relevant classes

6. File Handling — No Database Required

Students are not required to use SQLite, MySQL, PostgreSQL, or another database. The application must persist data using files.

• JSON (recommended)

• CSV

• TXT

The application must load existing data when it starts, allow data to be created/read/updated/deleted, support searching and filtering, and save changes before the application closes.

Recommended: Create a FileManager class to keep file handling separate from business logic.

7. GUI Requirements

GUI is mandatory. A console-only application will not be accepted as the final project.

Students are responsible for self-studying one GUI framework. Possible choices include:

• Tkinter

• CustomTkinter

• PyQt

• PySide

The GUI should include a main window/dashboard, navigation, forms, data tables, search/filter controls, and user feedback messages.

8. Dashboard

The application should include a dashboard displaying useful information using real application data, not hard-coded numbers.

Examples: Total Members, Active Memberships, Expired Memberships, Total Books, Available Books, Total Students, Active Courses, Sales, or Pending Tasks.

9. Testing

Each team must create at least 10 test scenarios covering:

• 3 successful scenarios

• 3 invalid input scenarios

• 2 business-rule violations

• 2 edge cases

Automated tests using pytest are welcome if students have learned them, but the minimum requirement is to document and execute the test scenarios.

10. Recommended Project Structure

Keep the structure simple and understandable. The following is recommended, but not mandatory:

project/
■■■ main.py
■■■ models/

■ ■■■ entity_1.py
■ ■■■ entity_2.py
■■■ services/
■ ■■■ service.py
■■■ data/
■ ■■■ data.json
■ ■■■ ...
■■■ file_manager/
■ ■■■ file_manager.py
■■■ gui/
■ ■■■ app.py
■■■ tests/
■ ■■■ test_scenarios.txt
■■■ README.md

11. Project 1 — Gym Management System

Main Classes: Member, Trainer, Membership, Payment, Attendance, Gym

Features:

• Add/update/delete members

• Add trainers

• Create and renew memberships

• Record payments and attendance

• Assign trainers

• Search members

• Filter active/expired memberships

• Dashboard

Business Rules:

• A member cannot attend if their membership is expired.

• A member cannot have two active memberships.

• Payment amount must be greater than zero.

• A member must exist before attendance can be recorded.

12. Project 2 — Library Management System

Main Classes: Book, User, Author, Borrowing, Category, Library

Features:

• Add/update/delete books

• Add users

• Search books

• Borrow and return books

• Track available copies

• Track borrowers

• Search by title/author

• Reports

Business Rules:

• A book cannot be borrowed if no copies are available.

• A user cannot borrow the same book twice.

• A user cannot exceed the maximum borrowing limit.

• A book must exist before it can be borrowed.

13. Project 3 — Academy Courses Management

Main Classes: Student, Instructor, Course, Enrollment, Payment, Attendance, Academy

Features:

• Add/update/delete students

• Add instructors

• Create/update courses

• Enroll/remove students

• Record attendance

• Track payments

• Search students/courses

• Course dashboard

Business Rules:

• A student cannot enroll in a full course.

• A student cannot enroll in the same course twice.

• A course must have an instructor.

• Payment amount must be greater than zero.

14. Project 4 — Restaurant Management

Main Classes: Customer, Table, MenuItem, Order, OrderItem, Payment, Restaurant

Features:

• Manage tables and menu

• Create orders

• Add/remove items

• Calculate totals

• Apply discounts

• Process payment

• Search orders

• Daily sales report

Business Rules:

• An occupied table cannot be assigned again.

• Quantity must be greater than zero.

• An empty order cannot be checked out.

• Order total is calculated automatically.

15. Project 5 — Clinic Management

Main Classes: Patient, Doctor, Appointment, MedicalRecord, Payment, Clinic

Features:

• Register patients/doctors

• Book/cancel appointments

• Search patients

• View schedule

• Record visits

• Track payments

Business Rules:

• A doctor cannot have two appointments at the same time.

• An appointment must belong to an existing patient.

• Appointment status: Scheduled, Completed, Cancelled.

16. Project 6 — Inventory Management

Main Classes: Product, Category, Supplier, Purchase, Sale, StockTransaction, Inventory

Features:

• Add/update/delete products

• Manage suppliers

• Record purchases/sales

• Update stock

• Search products

• Low-stock report

• Stock history

Business Rules:

• Stock cannot become negative.

• Sale quantity cannot exceed available stock.

• Product price must be positive.

• Product ID must be unique.

17. Project 7 — Car Rental System

Main Classes: Car, Customer, Rental, Payment, Branch, CarRental

Features:

• Add cars/customers

• Rent/return cars

• Calculate rental cost

• Search available cars

• Rental history

• Payment tracking

Business Rules:

• A rented car cannot be rented again.

• Return date cannot be before rental date.

• Rental cost depends on number of days.

18. Project 8 — Hotel Management

Main Classes: Guest, Room, Reservation, Payment, Service, Hotel

Features:

• Add rooms

• Register guests

• Create reservations

• Check-in/check-out

• Add services

• Calculate bill

• Search reservations

• Room availability

Business Rules:

• A room cannot have overlapping reservations.

• A guest cannot check out before check-in.

• Room cost depends on room type.

19. Project 9 — Personal Finance

Main Classes: User, Account, Income, Expense, Category, Budget, FinanceManager

Features:

• Add income/expenses

• Manage categories

• Create budgets

• Search transactions

• Filter by category

• Monthly summary

• Budget tracking

Business Rules:

• Transaction amount must be positive.

• Expense must have a category.

• Budget cannot be negative.

• Monthly income and expenses should be calculated automatically.

20. Project 10 — Small E-Commerce System

Main Classes: Customer, Product, Category, Cart, Order, OrderItem, Payment

Features:

• Product/customer management

• Shopping cart

• Add/remove items

• Checkout

• Calculate totals

• Discounts

• Stock management

• Order history

• Product search

Business Rules:

• Cannot purchase more than available stock.

• Cannot checkout an empty cart.

• Product price must be positive.

• Order total is calculated automatically.

21. Project 11 — Cinema Management

Main Classes: Movie, Customer, CinemaHall, Show, Ticket, Payment, Cinema

Features:

• Add movies

• Create shows

• Manage halls

• Display available seats

• Book/cancel tickets

• Search movies

• Calculate ticket price

• Booking history

Business Rules:

• A seat cannot be booked twice for the same show.

• A show must belong to a movie.

• Hall capacity cannot be exceeded.

22. Project 12 — Car Workshop

Main Classes: Customer, Vehicle, Mechanic, Service, Appointment, Invoice, Workshop

Features:

• Register customers/vehicles

• Create appointments

• Assign mechanics

• Record services

• Generate invoices

• Vehicle service history

• Search vehicles

Business Rules:

• A vehicle can have multiple service records.

• An appointment must have a mechanic.

• Invoice total is calculated automatically.

23. Project 13 — Event Management

Main Classes: Event, Participant, Organizer, Venue, Ticket, Payment, EventManager

Features:

• Create events

• Manage venues

• Register participants

• Sell tickets

• Track capacity

• Search/filter events

• Payment tracking

• Dashboard

Business Rules:

• Event cannot exceed venue capacity.

• Participant cannot register twice.

• Ticket price must be positive.

24. Project 14 — Pharmacy Management

Main Classes: Medicine, Category, Supplier, Customer, Sale, SaleItem, Pharmacy

Features:

• Add/update/delete medicines

• Manage suppliers

• Record sales

• Track inventory

• Search medicines

• Low-stock alerts

• Expiration tracking

• Sales report

Business Rules:

• Expired medicine cannot be sold.

• Sale quantity cannot exceed stock.

• Medicine price must be positive.

25. Project 15 — Task & Project Management

Main Classes: User, Team, Project, Task, Comment, ProjectManager

Features:

• Create projects/tasks

• Assign tasks

• Change status

• Set priorities

• Add comments

• Search/filter tasks

• Track deadlines

• Project dashboard

Business Rules:

• A task must belong to a project.

• A task must have a responsible user.

• Task status must be controlled.

• Deadline cannot be invalid.

26. Team Responsibilities

Team of 2

Student 1: Models, OOP, Business Logic, File Handling.
Student 2: Services, GUI, Testing.
Both students must understand the complete project.

Team of 3

Student 1: Models, OOP, Business Rules.
Student 2: File Handling, Services, Testing.

Student 3: GUI, Dashboard, User Experience.
All students must understand the complete project.

27. Deliverables

• Complete Python source code

• GUI application

• Data files

• README.md

• UML/Class Diagram

• At least 10 test scenarios

• Screenshots

• Presentation / Demo

28. README Requirements

• Project Name

• Team Members

• Project Description

• Main Features

• Classes

• OOP Concepts Used

• Business Rules

• File Storage

• GUI Framework

• How to Run

• Test Scenarios

• Screenshots

29. UML / Class Diagram

Create a simple class diagram showing classes, attributes, methods, inheritance and relationships.

Students may self-study a suitable UML diagram tool.

30. What You Should NOT Do

• No SQLite, MySQL, PostgreSQL or other database is required.

• No external APIs are required.

• Do not submit a console-only final project.

• Do not copy-paste a project you cannot explain.

• Do not put the entire system into one huge class.

• Do not create meaningless inheritance just to satisfy a requirement.

31. Final Presentation

• Explain the real-world problem.

• Demonstrate the GUI.

• Explain classes and objects.

• Explain constructors.

• Demonstrate encapsulation.

• Demonstrate inheritance.

• Demonstrate polymorphism.

• Explain abstraction.

• Explain composition.

• Demonstrate file loading and saving.

• Demonstrate at least 3 business rules.

• Demonstrate error handling.

32. Evaluation

Category	Weight
OOP Design & Implementation	25%
Business Logic	20%
GUI	20%
File Handling & Persistence	10%
Validation & Exception Handling	10%
Testing	5%
Type Hints & Code Quality	5%
Documentation & Presentation	5%
Total	100%

33. Main Learning Goal

The project should demonstrate that you can take a simple real-world problem and transform it into a working Python application.

Real Problem → Requirements → Class Design → OOP → Business Logic → File Storage → GUI → Testing

The objective is not to build a huge system. Build a small, complete and usable application that demonstrates strong OOP thinking and good software structure.

Instant

Project Requirements prepared by **Mohamed Abu Obaida**